CMFD v3 — Pure-QNN Architecture Brief

What the current Swap-Test Siamese QNN pipeline actually does, distilled from the last two working sessions. For your own reading / brainstorming before the next round.

---

1. The end-to-end pipeline, stage by stage

Code: **code/cmfd_swaptest_pipeline.py** (model, training, CoMoFoD eval), **code/inference.py** (Streamlit-facing wrapper), **code/train_split.py** (train/test-split trainer).

Stage	Classical / Quantum	What it does
Load + grayscale + resize	Classical	PIL load, .convert('L'), LANCZOS resize to a fixed working size
Sliding-window patch extraction	Classical	extract_all_patches() — pure geometric tiling, no content judgement
State preparation	Classical arithmetic (NOT feature extraction)	prepare_patch_state() — flatten + L2-normalize + zero-pad to 2^n. Every pixel value is preserved.
Amplitude encoding	Quantum	qml.AmplitudeEmbedding loads the normalized patch vector into a register (n = log2(pixel count) qubits)
Data re-uploading ansatz	Quantum (trained)	Tied RX/RY/RZ + CNOT layers, weights shared across both patch registers (Siamese)
Offset register	Classical geometry -> Quantum	encode_offset(dy,dx) angle-encoded, trainable CRY couples it into the ancilla readout
Swap test	Quantum	H -> CSWAP(ancilla, a_k, b_k) for all k -> CRY -> H; measures on the ancilla
Threshold + mask paint	Classical decision	score >= sim_threshold -> paint both patches into the mask (mask_from_scores)
Connected components -> boxes	Classical, display-only	inference.py — runs AFTER the decision, feeds nothing back into it

2. Is this doing feature extraction? No.

This was the central question across both sessions. **v2** (retired) fed the circuit a DCT + intensity-histogram descriptor — a human-chosen basis transform that decided which 24 numbers mattered before the circuit ever ran. That *is* feature extraction, and the module docstring names it as the reason v2 could not honestly be called ‘pure quantum’.

v3 (current) removed this entirely. prepare_patch_state() only flattens and L2-normalizes — no basis change, no projection, no discarded information. This is **state preparation** (the classical-to-quantum data-loading step every QML algorithm needs), not feature extraction. The distinction that matters: feature extraction decides *what is informative* and throws the rest away; normalization just rescales a vector so its squared amplitudes sum to 1 (the unit-norm condition every amplitude-encoded quantum state must satisfy). Nothing is judged, nothing is discarded.

Bottom line: the circuit sees the complete raw patch. No DCT, no Zernike moments, no hand- or algorithm-chosen descriptor exists anywhere in v3.

3. Classical steps that remain — unavoidable vs. design choice

Classical step	Category	Why
Image file decode	Unavoidable	No quantum file-decoder exists; data starts as classical bits
RGB -> grayscale	Design choice	Fewer values per patch -> fewer qubits (256 vs 768 for 16x16). Discards color, a real potential forgery cue
Resize to working res.	Forced by tractability	Fixes patch count, which fixes O(N^2) pair-scoring cost (256px tiled ~5h/image per README)
Patch extraction (not whole-image encoding)	Design choice	A single whole-image FRQI/NEQR state would need ~16+ qubits just for position at 256x256 - impractical to simulate
L2-normalize + zero-pad	Unavoidable	The amplitude-encoding contract itself, not a content decision
(dy,dx) -> angle	Unavoidable	Coordinates are classical numbers; must enter the circuit as some encoding
Threshold on measured	Unavoidable	Every quantum algorithm ends in a classical readout of a measurement - this is reading the answer, not engineering a feature

4. Qubit count: amplitude encoding vs. FRQI/NEQR

One recurring point of confusion worth locking down. Under the **amplitude encoding** this code actually uses (qubits_for_patch()), qubit count = ceil(log2(number of pixel values)) - a pixel's intensity becomes an *amplitude* (a continuous real number), costing zero extra qubits regardless of bit-depth.

- Grayscale, 16x16 patch: 256 values -> **8 qubits**
- RGB, 16x16 patch: 768 values -> **10 qubits** (needs MORE qubits than grayscale, since there are 3x more scalar values)

This is the opposite relationship to **FRQI/NEQR** (the other “quantum image representation” schemes from the original brainstorm, not implemented here). Those use *basis* encoding for intensity - NEQR needs a dedicated intensity register of ceil(log2(gray levels)) qubits (8 extra qubits for 256 levels vs. 1 for a binary image). Under FRQI/NEQR, “grayscale needs more qubits” is true. Under amplitude encoding (what's running), it's the reverse. Worth keeping straight for the report - they are not interchangeable encoding schemes.

5. Robustness: what's handled, what isn't

The known hazard with comparing raw pixel amplitudes directly: brightness, rotation, scaling, JPEG compression, and noise all perturb the amplitude vector. v3's answer is **not** a robust classical descriptor - it's training-time augmentation acting on a *trainable* circuit, so invariance is learned inside rotation angles instead of engineered classically.

Threat	Status	Mechanism
Brightness / contrast	Partially handled	_degrade_patch(): random contrast/brightness on synthetic positives AND negatives
Sensor noise	Partially handled	_degrade_patch(): Gaussian noise
JPEG compression	Partially handled	_degrade_patch(): real JPEG round-trip at random quality
Soft/blended paste edge	Partially handled	_feather_paste(): alpha-blended boundary instead of a hard seam
Rotation	NOT handled (off by default)	rot90 permutes the amplitude vector near-orthogonal (~0.01 vs ~1.0 for an exact copy); offset register is translation-only, so rotated training targets are unfittable
Scale	NOT handled at all	No scale augmentation or scale-aware register anywhere in the pipeline

6. Where the numbers actually stand (measured, not guessed)

- **Training loss** (checkpoints_pure/history.json, 10 epochs, vanilla SGD): bounces 0.033-0.056, no downward trend - a fixed-step optimizer oscillating on a noisy variational loss surface, not necessarily a broken architecture.
- **Test metrics** (results_coverage_tuned/results_grip.csv, 3 GRIP images): recall ≈ 1.0, precision 0.10-0.34 - the model is flagging almost every pair as a match. sim_threshold (0.8-0.9) was a guess, never swept against a PR curve.
- **Documented root cause** (module docstring): the offset register is a *weaker* substitute for v2's classical Hough-style vote - offset consistency is a global property of the pair ensemble that a per-pair circuit can't see. This is the leading suspect for the false-positive rate.

7. Already implemented this session (Tier 0 / Tier 1)

Change	File	Why
score_all_pairs() / mask_from_scores() split	cmfd_swaptest_pipeline.py	Lets a threshold sweep reuse ONE circuit pass per image instead of re-running O(N^2) scoring per candidate threshold
calibrate_threshold.py (new)	code/	Sweeps sim_threshold against real GT on GRIP/COVERAGE, reports mean-F1 per threshold + best pick
Adam optimizer (default)	train()	Replaces fixed-step GradientDescentOptimizer, the likely cause of the flat/oscillating training loss
Batched training cost	train()	One broadcasted QNode call per image-batch instead of a Python loop over pairs - fewer dispatches, smoother gradient estimate
Held-out validation loss	train()	Per-epoch val loss (val_images, default 4) on data disjoint from training - lets you pick the best epoch instead of trusting noisy training loss alone

None of this touches the circuit's decision logic or reintroduces feature extraction - it's optimizer/calibration hygiene on top of the existing pure architecture.

8. Open options to brainstorm (nothing implemented yet)

A. Rotation / scale robustness without reintroducing feature extraction

Instead of Zernike moments (a classical descriptor - rejected), try candidate patch B at a few rotations/scales (rot90 family; a couple of LANCZOS rescales), amplitude-encode each, let the circuit score all of them, keep the max. rot90 is a lossless pixel relabeling, not a feature decision - same ‘classical scheduling, quantum decision’ framing already used for min_spatial_dist and the offset register. Cost: multiplies circuit calls per pair by the number of candidates tried, stacking with the existing O(N^2) latency problem.

B. Classical offset-consistency post-filter (precision fix, paused pending your decision)

Cluster surviving matched pairs by (dy,dx) after thresholding; only paint clusters above a minimum size into the final mask - restores the spirit of v2's Hough vote as a post-hoc filter that never touches the circuit's similarity decision. Needs to be reported honestly as a classical spatial-consistency stage layered on top of a quantum decision, not folded into the ‘pure detection’ claim without comment.

C. Latency (decision already made: stay strictly pure)

- Shrink patch_size (fewer qubits -> exponentially cheaper simulation) at the cost of spatial precision on small forgeries.
- Push --batch-size higher in inference (parameter-broadcasting, already measured ~1.3x).
- Not chosen: classical PatchMatch-style candidate pruning to cut O(N^2) to ~O(N) - bigger speedup, but needs careful framing since it restricts what the circuit even gets to see.

D. Still-open engineering backlog

- Best-checkpoint picker: evaluate every epoch's checkpoint on held-out real images and select by F1, instead of always shipping the last epoch (epoch 9 beat epoch 10 on training loss in the existing run).
- decision_pairs (the image-level FORGED/AUTHENTIC cutoff in inference.py) is a separate threshold from sim_threshold and has not been calibrated by calibrate_threshold.py yet.
- Grayscale-vs-RGB ablation: color correlation between source and pasted region is a real forgery cue currently discarded before the circuit ever runs.

Generated from the CMFD Phase-2 project working sessions on threshold calibration, training stability, and the feature-extraction/state-preparation distinction. Code references: code/cmfd_swaptest_pipeline.py, code/inference.py, code/train_split.py, code/calibrate_threshold.py, code/grip_dataset.py.